

9. Serialization

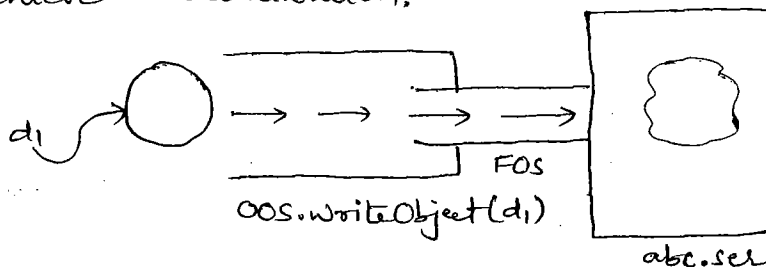
1. Introduction
2. Object graphs in Serialization
3. Customized Serialization
4. Serialization w.r.t inheritance
5. Externalization
6. SerialVersionUID.

1. Introduction :-

Serialization :-

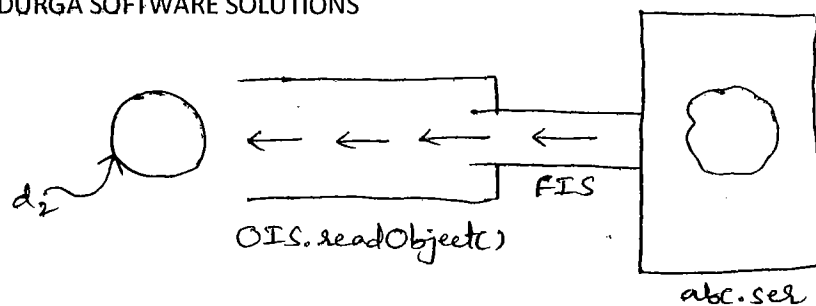
- The processing of writing state of an object to a file is called Serialization. But strictly speaking, it is the process of converting an object from Java supported form to either File supported form or Network supported form.
- By using FileOutputStream and ObjectOutputStream classes we can achieve Serialization.

Ex:



Deserialization :-

- The process of reading state of an object from a file is called Deserialization. But strictly speaking it is the process of converting an object from either file or network supported form into Java supported form.
- By using FileInputStream and ObjectInputStream classes we can achieve Deserialization.

ex:ex:

import java.io.*;

class Dog implements Serializable

{ int i=10;

int j=20;

}

class SerializeDemo

{

public void m() throws Exception

{

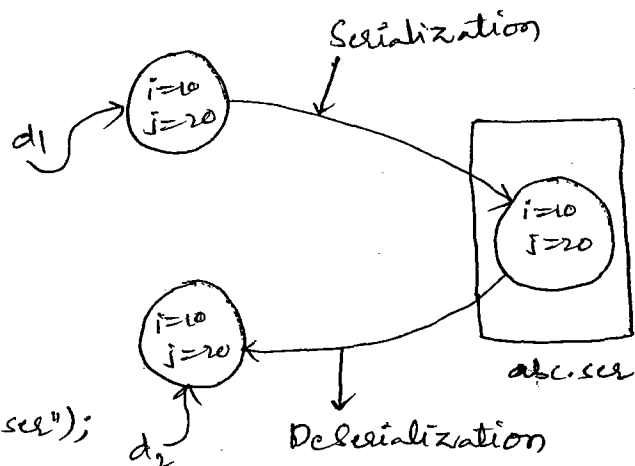
Dog d1 = new Dog();

Serialization {

```
FOS fos = new FOS("abc.ser");
OOS oos = new OOS(fos);
oos.writeObject(d1);
```

Deserialization {

```
FIS fis = new FIS("abc.ser");
OIS ois = new OIS(fis);
Dog d2 = (Dog) ois.readObject();
S.o.p(d2.i + "... " + d2.j);
O/P: 10 ... 20
```



→ We can serialize only Serializable objects.

→ An object is said to be Serializable iff the corresponding class implements Serializable interface.→ Serializable interface present in java.io package & it doesn't contain any methods. It is a marker interface.

→ If we are trying to serialize a non-serializable object then we will get runtime exception saying NotSerializableException.

transient keyword:-

→ transient is the modifier applicable only for variables.

→ While performing serialization if we don't want to save the value of a particular variable to meet security constraints such type of variables we have to declare with transient keyword.

→ At the time of serialization JVM ignores original value of transient variables and saves default value to the file.

→ Hence transient means not to serialize.

static vs transient:-

→ static variable is not part of object state. Hence it won't participate in serialization.

→ Due to this declaring static variable as transient there is no use.

final vs transient:-

→ final variables will be participated in serialization directly by their values.

→ Due to this declaring final variable as transient there is no use.

Declaration	Output
int i=10; int j=20;	10...20
transient int i=10; int j=20;	0...20..

transient static int i=10; transient int j=20;	10 . . . 0
transient int i=10; transient final int j=10;	0 . . . 10
transient static int i=10; transient final int j=20;	10 . . . 20

→ We can serialize multiple objects to the file. But in which order we serialize in the same order only deserialize.

Ex: Dog d₁ = new Dog();
Cat c₁ = new Cat();
Rat r₁ = new Rat();
FOS fos = new FOS("abc.ser");
OOS oos = new OOS(fos);
oos.writeObject(d₁);
oos.writeObject(c₁);
oos.writeObject(r₁);

FIS fis = new FIS("abc.ser");
OIS ois = new OIS(fis);
Dog d₂ = (Dog) ois.readObject();
Cat c₂ = (Cat) ois.readObject();
Rat r₂ = (Rat) ois.readObject();

→ If we don't know order of objects in Serialization

Ex: FIS fis = new FIS("abc.ser");
OIS ois = new OIS(fis);
Object o = fis.readObject();

```

if (o instanceof Dog)
{
    Dog d = (Dog) o;
    // perform Dog specific functionality
}
else if (o instanceof Cat)
{
    Cat c = (Cat) o;
    // perform Cat specific functionality
}
}

```

2. Object Graphs in Serialization:—

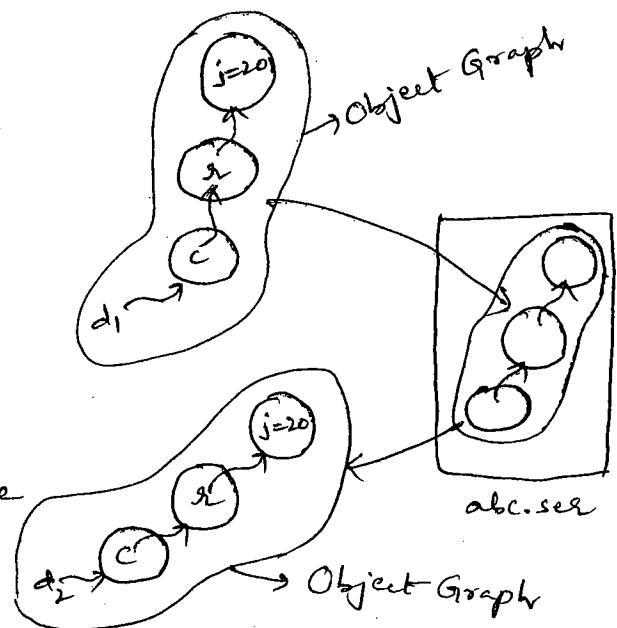
→ Whenever we are serializing an object the set of all objects which are reachable from that object will be Serialized automatically. This group of objects is nothing Object Graph in Serialization.

→ In Object graph, every object should be Serializable. If at least one object is not Serializable then we will get RE saying NotSerializableException.

```

ex: import java.io.*;
class Dog implements Serializable
{
    Cat c = new Cat();
}
class Cat implements Serializable
{
    Rat r = new Rat();
}
class Rat implements Serializable
{
    int j = 20;
}

```



```

class SerializedDemo1
{
    public void m() throws Exception
    {
        Dog d1 = new Dog();
        FOS fos = new FOS("abc.ser");
        OOS oos = new OOS(fos);
        oos.writeObject(d1);

        FIS fis = new FIS("abc.ser");
        OIS ois = new OIS(fis);
        Dog d2 = (Dog)ois.readObject();
        S.o.p(d2.c.e.j); // => O/P : 20
    }
}

```

- In the above example, whenever we are serializing Dog object automatically Cat and Rat objects will be serialized becoz these are part of Object graph of Dog object.
- Among Dog, Cat & Rat if atleast one object is non-serializable then we will get RE saying NotSerializableException.

3. Customized Serialization:-

- During default Serialization there may be a chance of loss of information due to transient keyword.

```

Ex: import java.io.*;
    class Account implements Serializable
    {
        String username = "durga";
        transient String pwd = "anushka";
    }

```

```
class CustSerializeDemo
```

```
{
```

```
    P.S.V.M.C() throws Exception
```

```
{
```

```
    Account a1 = new Account();
```

```
    S.o.p(a1.username + "... " + a1.pwd); ⇒ O/P: durga ... anushka
```

```
    FOS fos = new FOS("abc.ser");
```

```
    OOS oos = new OOS(fos);
```

```
    oos.writeObject(a1);
```

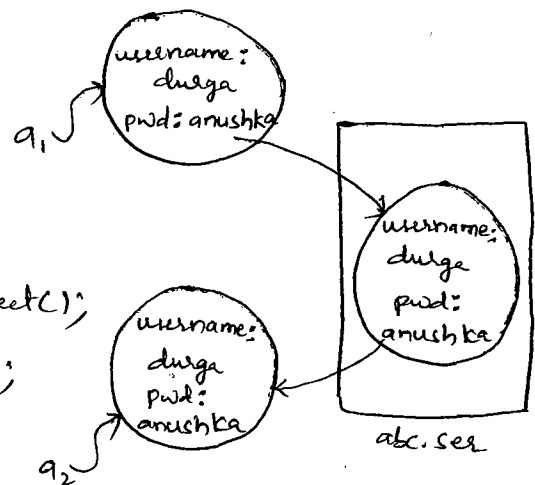
```
    FIS fis = new FIS("abc.ser");
```

```
    OIS ois = new OIS(fis);
```

```
    Account a2 = (Account) ois.readObject();
```

```
    S.o.p(a2.username + "... " + a2.pwd);
```

```
} } O/P: durga ... null
```



- In the above example, before Serialization Account object can provide proper username and pwd. But after Deserialization Account object can provide only username, but not pwd.
- This is due to declaring pwd variable as transient.
- Hence during default Serialization there may be a chance of loss of information due to transient keyword.
- To recover this loss of information we should go for Customized Serialization.
- We can implement Customized Serialization by using the following 2 methods.

①. `private void writeObject(ObjectOutputStream oos) throws Exception`

→ This method will be executed automatically at the time of Serialization. Hence while performing Serialization if we want to do any extra work we have to write code in this method only.

② `private void readObject(ObjectInputStream ois) throws Exception`

→ This method will be executed automatically at the time of Deserialization. Hence while performing Deserialization if we want to do any extra work we have to define that in this method only.

→ While performing which object Serialization we have to do this extra work in the corresponding class we have to define the above methods.

For Example, while performing Account object Serialization if we required to do extra work then in Account class we have to define above methods.

```
Ex: ① import java.io.*;
    class Account implements Serializable
    {
        String username = "durga";
        transient String pwd = "anushka";
        private void writeObject(OOS os) throws Exception
        {
            os.defaultWriteObject();
            String epwd = "123" + pwd;
            os.writeObject(epwd);
        }
    }
```

```
private void readObject(OIS is) throws Exception
```

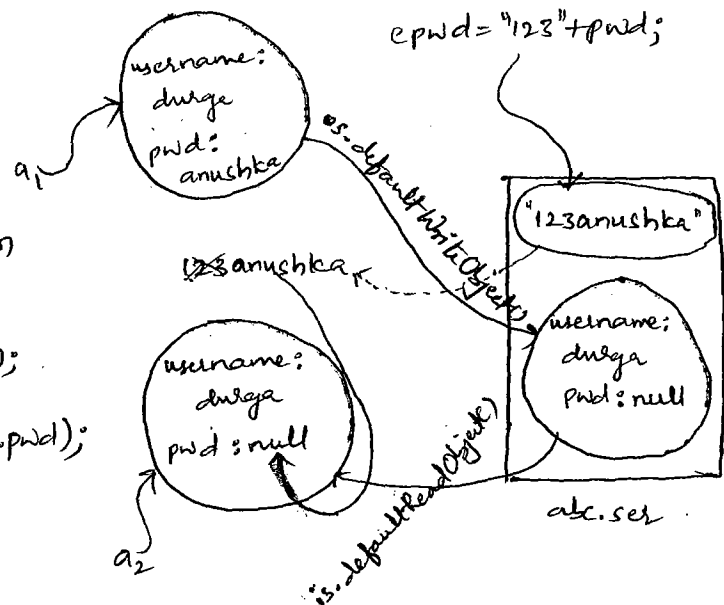
```
{
    is.defaultReadObject();
    String epwd = (String) is.readObject();
    pwd = epwd.substring(3);
}
```

```
}
class CustSerializeDemo1
{
    p s v m (-) throws Exception
    {
```

```
Account a1 = new Account();
S.o.p(a1.username + "... " + a1.pwd);
FOS fos = new FOS("abc.ser");
OOS oos = new OOS(fos);
oos.writeObject(a1);

FIS fis = new FIS("abc.ser");
OIS ois = new OIS(fis);
Account a2 = (Account) ois.readObject();
S.o.p(a2.username + "... " + a2.pwd);
}
```

o/p: durga... anushka
durga... anushka



```
ex2: import java.io.*;
class Account implements Serializable
{
    String username = "durga";
    transient String pwd = "anushka";
```

```

transient int pin = 1234;
private void writeObject (OOS os) throws Exception
{
    os.defaultWriteObject();
    String epwd = "123" + pwd;
    os.writeObject(epwd);
    int epin = pin + 4444;
    os.writeObject(epin);
}

private void readObject (OIS is) throws Exception
{
    is.defaultReadObject();
    String epwd = (String) is.readObject();
    pwd = epwd.substring(3);
    int epin = is.readInt();
    pin = epin - 4444;
}
}

```

4. Serialization w.r.t Inheritance :-

Case (i): If parent is Serializable then by default every child is Serializable i.e., Serializable nature is inheriting from parent to child. Hence even though child class doesn't implement Serializable if parent class implements Serializable then we can serialize child class object.

Ex: `import java.io.*;`
 class Animal implements Serializable
 {
 int i = 10;
 }

```

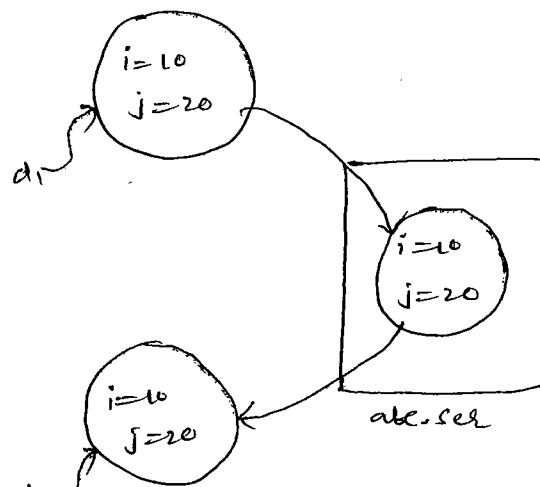
class Dog extends Animal
{
    int j=20;
}

class InSerializeDemo
{
    public static void main() throws Exception
    {
        Dog d1 = new Dog();
        FOS fos = new FOS("abc.ser");
        OOS oos = new OOS(fos);
        oos.writeObject(d1);

        FIS fis = new FIS("abc.ser");
        OIS ois = new OIS(fis);
        Dog d2 = (Dog) ois.readObject();
        S.o.p(d2.i + "... " + d2.j);
    }
}

```

o/p: 10 ... 20



Case ii):

1. Even though parent class doesn't implement Serializable interface we can serialize child class object if child class implements Serializable. i.e.,
2. At the time of Serialization JVM will check is any instance variable is inheriting from non-Serializable parent or not. If any variable is inheriting from non-Serializable parent then JVM ignores original value & save default value to the file.
3. At the time of Deserialization JVM will check is any parent class is non-Serializable or not. If any parent class is non-Serializable then execute Instance Control Flow in that.

non-Serializable parent & share its instance variables to the current object.

4. In Instance Control Flow execution of non-Serializable parent JVM will always invoke no-argument constructor. Hence every non-Serializable class should compulsorily contain no-argument constructor, o.w we will get RE saying InvalidClassException.

Ex: import java.io.*;

```
class Animal
```

```
{
```

```
    int i=10;
```

```
    Animal()
```

```
{
```

```
        s.o.p("Animal constructor called");
```

```
}
```

```
}
```

```
class Dog extends Animal implements Serializable
```

```
{
```

```
    int j=20;
```

```
    Dog()
```

```
{
```

```
        s.o.p("Dog constructor called");
```

```
}
```

```
}
```

```
class EnSerializeDemo1
```

```
{
```

```
    p s v mC-)
```

```
{
```

```
        Dog d1=new Dog();
```

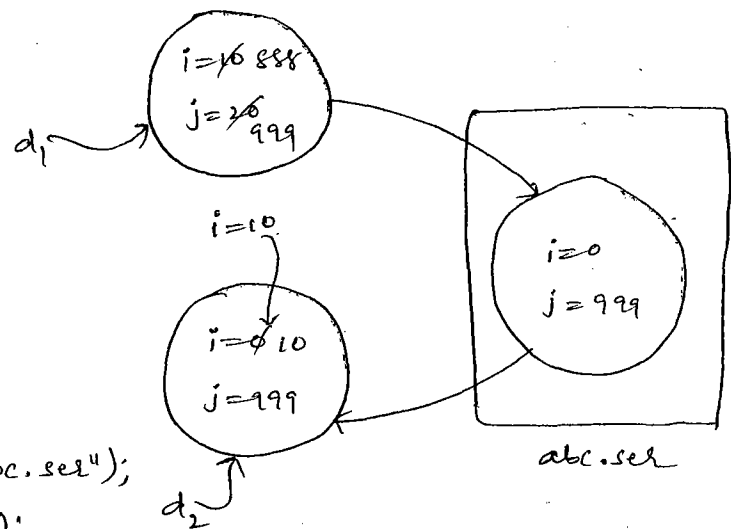
```
        d1.i=888;
```

```
        d1.j=999;
```

```
        FOS fos=new FOS("abc.ser");
```

```
        OOS oos=new OOS(fos);
```

```
        oos.writeObject(d1);
```



```

S.o.p ("Deserialization Started");
FIS fis = new FIS ("abc.ser");
OIS ois = new OIS (fis);
Dog d2 = (Dog) ois.readObject();
S.o.p (d2.i + "... " + d2.j);
}

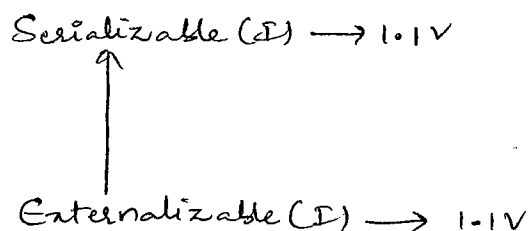
```

o/p: Animal constructor called
Dog constructor called
Deserialization Started
Animal constructor called
10... 999

*** }

5. Externalization:—

- In Serialization, everything takes care by JVM & programmer doesn't have any control.
- In Serialization, total object will be serialized always & it is not possible to serialize part of the object, which may create performance problems in some cases.
- To overcome these problems we should go for Externalization, where everything takes care by programmer & JVM doesn't have any control.
- The advantage of Externalization is based on our requirement we can save either total object or part of the object. So that relatively performance will be improved.
- To provide Externalizable ability for any Java object compulsory the corresponding class should implement Externalizable interface.
- Externalizable interface is the child interface of Serializable & it contains 2 methods are writeExternal() & readExternal()



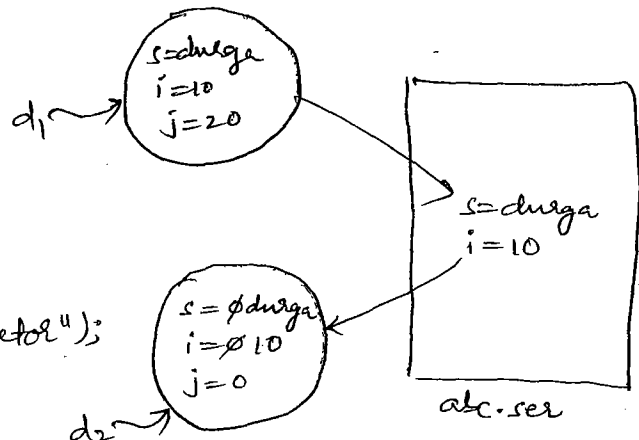
①. `public void writeExternal(ObjectOutput out) throws IOException`

- This method will be executed automatically at the time of Serialization
- Within this method we have to write code to save required variables to the file.

②. `public void readExternal(ObjectInput in) throws IOException, ClassNotFoundException`

- This method will be executed automatically at the time of Deserialization.
- Within this method we have to write code to read required variables from the file & assign to the current object.
- Strictly speaking at the time of Deserialization JVM will create a separate new object by executing public no-argument constructor, on that object `readExternal()` method will be executed.
- Externalizable class should compulsory contains public no-argument constructor otherwise, we will get RE saying InvalidClassException.

ex: `import java.io.*;`
`public class ExternalizableDemo implements Externalizable`
`{`
`String s;`
`int i;`
`int j;`
`public ExternalizableDemo`
`{`
`S.o.p("public no-arg constructor");`
`}`



```

public ExternalizableDemo (String s, int i, int j)
{
    this.s = s;
    this.i = i;
    this.j = j;
}

public void writeExternal (ObjectOutput out) throws IOException
{
    out.writeObject(s);
    out.writeInt(i);
}

public void readExternal (ObjectInput in) throws IOException,
                                                                    ClassNotFoundException
{
    s = (String) in.readObject();
    i = in.readInt();
}

public static void main (-) throws Exception
{
    ExternalizableDemo d1 = new ExternalizableDemo ("durga", 10, 20);
    FOS fos = new FOS ("abc.ser");
    OOS oos = new OOS (fos);
    oos.writeObject(d1);

    FIS fis = new FIS ("abc.ser");
    OIS ois = new OIS (fis);
    ExternalizableDemo d2 = (ExternalizableDemo) ois.readObject();
    S.o.p (d2.s + "... " + d2.i + "... " + d2.j);
}

```

→ If the class implements Externalizable interface then the o/p is.

public no-arg constructor
 durga ... 10 ... 0

→ If the class implements Serializable interface then the
o/p is durga...10...20

Note:- In Externalization, transient keyword won't play any role
 ofcourse it is not required.

Differences b/w Serialization and Externalization:-

Serialization	Externalization
1. It is meant for default Serialization. 2. Here everything takes care by JVM and programmer doesn't have any control. 3. In Serialization, total object will be serialized always & it is not possible to serialize part of the object. 4. Relatively performance is low. 5. Serialization is the best choice if we want to save total object to the file. 6. Serializable interface doesn't contain any methods & it is a marker interface.	1. It is meant for customized Serialization. 2. Here everything takes care by programmer and JVM doesn't have any control. 3. In Externalization, based on our requirement we can save either total object or part of the object. 4. Relatively performance is high. 5. Externalization is the best choice if we want to save part of the object to the file. 6. Externalizable interface contains 2 methods. <code>writeExternal()</code> and <code>readExternal()</code>. It is not a marker interface.

Serialization

7. Serializable class is not required to contain public no-argument constructor.

8. transient keyword will play role in Serialization.

Externalization

7. Externalizable class should compulsory contain public no-argument constructor o.w, we will get RE saying

InvalidClassException.

8. transient keyword won't play any role in Externalization.

6. serialVersionUID:-

- To perform Serialization & Deserialization internally JVM will use a unique identifier, which is nothing but serialVersionUID.
- At the time of Serialization JVM will save serialVersionUID.
- At the time of Deserialization JVM will compare serialVersionUID's and if it is matched then only the object will be deserialized o.w, we will get RE saying InvalidClassException.

The problems in depending on Default serialVersionUID:-

1. After Serialization if we change class file at server side then we can't perform Deserialization becoz of mismatch in serialVersionUID's of local class and serialized object.

In this case, at the time of Deserialization we will get RE saying InvalidClassException.

2. Both Sender and Receiver should use same version of JVM, if there is any incompatibility in JVM versions then Receiver is unable to deserialize.

In this case, also, Receiver will get RE saying InvalidClassException.

3. To generate serialVersionUID internally JVM will use complex algorithm, which may create performance problems.

→ We can solve above problems by configuring our own serialVersionUID.

→ We can configure serialVersionUID as follows.

```
private static final long serialVersionUID;
```

Ex: import java.io.*;

class Dog1 implements Serializable

```
{
    private static final long serialVersionUID = 1L;
```

```
    int i = 10;
```

```
    int j = 20;
}
```

Sender.java:-

```
import java.io.*;
```

```
class Sender
```

```
{
    p s v m (-) throws Exception
```

```
{
    Dog1 d1 = new Dog1();
```

```
    FOS fos = new FOS("abc.ser");
```

```
    OOS oos = new OOS(fos);
```

```
    oos.writeObject(d1);
}
```

Receiver.java:-

```
import java.io.*;
```

```
class Receiver
```

```
{
```

```
p < v m() throws Exception  
{  
    FIS fis = new FIS("abc.ser");  
    OIS ois = new OIS(fis);  
    Dog d2 = (Dog) ois.readObject();  
    S.o.p(d2.i + "... " + d2.j);  
}  
    o/p: 10... 20.
```

→ In the above program, after Serialization even though if we are performing any change to the .class file we can deserialize object.

→ If we configure our own serialVersionUID both Sender & Receiver are not required to maintain same JVM versions.

Note ①:- Some IDE's explicitly prompt the programmer to enter serialVersionUID.

② some IDE's explicitly generates serialVersionUID automatically instead of depending on JVM generated default serialVersionUID.

